

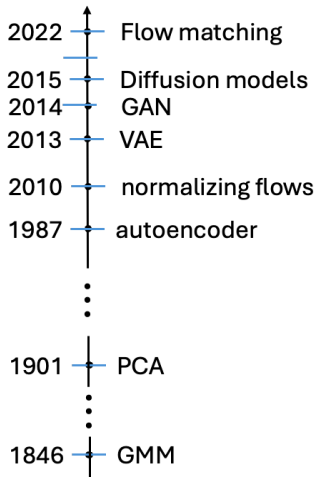
COMM114: Generative AI - GAN (part 1)

Guoqiang Zhang

September 2025

GAN

Generative adversarial network (GAN) was invented in 2014 by Ian Goodfellow and his colleagues in June 2014.



GAN

GAN is a powerful framework for implicitly and effectively learning the data distribution $p_{data}(\cdot)$ from a dataset $\{\mathbf{x}_i\}_{i=1}^n$.



Figure: Example of fake images generated by a pre-trained GAN (named styleGAN)

Objective of GAN

- **Input:** data samples $\{\mathbf{x}_i\}_{i=1}^n$.



- **Assumption:** Assume that the data samples are drawn from a certain data distribution $p_{data}(\cdot)$.
- **Objective of GAN:** To learn a nonlinear function $G_\phi(\cdot)$ (e.g., a deep neural network) that maps a Gaussian noise $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ to a sample $G_\phi(\mathbf{z})$ that looks as if it came from the data distribution $p_{data}(\mathbf{x})$.
- In the ideal case, the nonlinear function G_ϕ implicitly transforms a simple tractable distribution (e.g., $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$) to the data distribution $p_{data}(\mathbf{x})$.

Objective of GAN (cont.)

- The Generator maps a Gaussian noise $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ to a sample $G_\phi(\mathbf{z})$ that looks as if it came from the data distribution $p_{data}(\mathbf{x})$

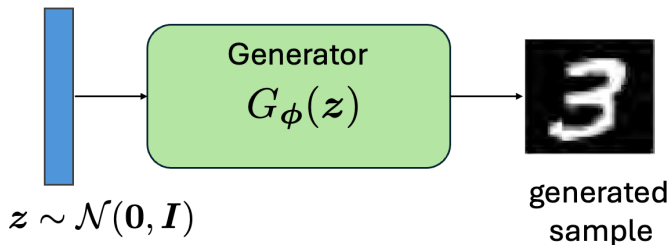
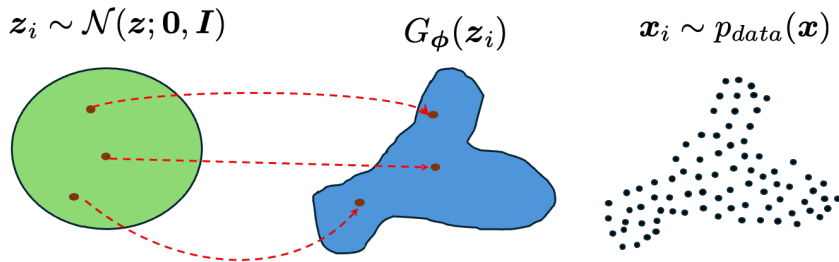


Figure: Demonstration of Generator $G_\phi(\cdot)$

Objective of GAN (cont.)

- In the ideal case, the distribution of $G_\phi(\mathbf{z}_i)$, where $\mathbf{z}_i \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ is the same as $p_{data}(\mathbf{x})$.



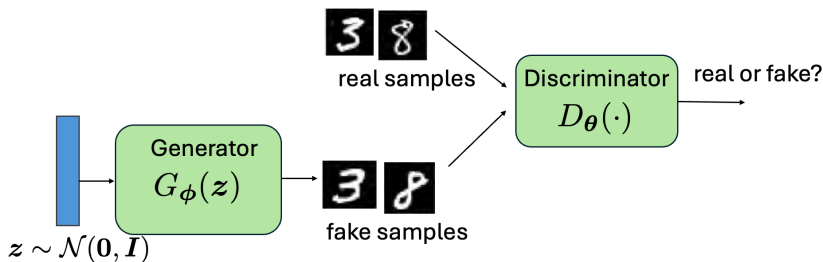
Question:

How to train the generator $G_\phi(\cdot)$?

GAN

GAN introduces a discriminator $D_{\theta}(\cdot)$ to help training the generator $G_{\phi}(\cdot)$.

- In the training process, the generator updates its parameter ϕ by attempting to produce more realistic (fake) samples that makes the discriminator thinks that the samples came from the real data distribution.
- The discriminator updates its parameter θ by distinguishing between real and fake samples more accurately.



Remark: $D_{\theta}(\cdot)$ is a scalar function since the discriminator solves a binary (real versus fake samples) [classification problem](#).

Updating discriminator in GAN

Guidance

- We let the output $D_{\theta}(\cdot)$ be a score between 0 and 1 representing the probability of its input sample being real or fake.
- A high value of $D_{\theta}(\cdot)$ indicating that the input sample has a high probability of being real, and vice versa.
- For a real sample $\mathbf{x}_i \sim p_{data}(\mathbf{x})$, the discriminator attempts to maximize the log likelihood $\log D_{\theta}(\mathbf{x}_i)$ as the discriminator already knows it is a real sample.
- For a given noise sample $\mathbf{z}_i$, the discriminator attempts to minimize the log likelihood $\log D_{\theta}(G_{\phi}(\mathbf{z}_i))$, or alternatively maximize $\log(1 - D_{\theta}(G_{\phi}(\mathbf{z}_i)))$ as the discriminator already knows it is a fake sample.
- Therefore, the discriminator can be updated via

$$\arg \max_{\theta} \left[\mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log \underbrace{D_{\theta}(\mathbf{x})}_{\text{push towards 1}} \right] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \left[\log(1 - \underbrace{D_{\theta}(G_{\phi}(\mathbf{z}))}_{\text{push towards 0}}) \right] \right].$$

Updating generator in GAN

Guidance

- We let the output $D_{\theta}(\cdot)$ be a score between 0 and 1 representing the probability of its input sample being real or fake.
- A high value of $D_{\theta}(\cdot)$ indicating that the input sample has a high probability of being real, and vice versa.
- For a given noise sample $\mathbf{z}_i$, the generator attempts to maximize the log likelihood $\log D_{\theta}(G_{\phi}(\mathbf{z}_i))$, or alternatively minimize $\log(1 - D_{\theta}(G_{\phi}(\mathbf{z}_i)))$
- Therefore, the generator can be updated via

$$\arg \min_{\phi} \left[\mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \log(1 - \underbrace{D_{\theta}(G_{\phi}(\mathbf{z}))}_{\text{push towards 1}}) \right].$$

Minmax optimization in GAN

The overall optimization in a GAN by putting the objectives of the generator and discriminator together is

$$\arg \min_{\phi} \max_{\theta} \left[\underbrace{\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_{\theta}(\mathbf{x})]}_{\text{1st term}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))}_{\text{2nd term}} \right].$$

- The 1st term is independent of the generator.
- the 2nd term involves both the generator and discriminator.
- The discriminator wants to maximize the second term whereas the generator wants to minimize it.

Two-player game when training a GAN

- The discriminator, as a binary classifier, attempts to distinguish the fake and real samples accurately.
- The generator attempts to fool the discriminator by generating more realistic (fake) samples.

Minmax optimization in GAN (cont.)

The overall optimization in a GAN by putting the objectives of the generator and discriminator together is

$$\arg \min_{\phi} \max_{\theta} \left[\underbrace{\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_{\theta}(\mathbf{x})]}_{\text{1st term}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))}_{\text{2nd term}} \right]. \quad (1)$$

- The 1st term is independent of the generator.
- The 2nd term involves both the generator and discriminator.
- The discriminator wants to maximize the second term whereas the generator wants to minimize it.

Two-player game when training a GAN

- Discriminator and generator should be updated sequentially rather than simultaneously.

Why simultaneous updates are generally avoided?

- **Convergence issues:** Simultaneous updates of the generator and discriminator can lead to instability and make it harder for the GAN to converge to a good solution.

Training procedure

Algorithm 1 GAN Training

- 1: **for** number of training iterations **do**
- 2: **for** for k steps **do**
- 3: Sample minibatch of m noise samples $\{\mathbf{z}_1, \dots, \mathbf{z}_m\}$ from $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- 4: Sample minibatch of m real data samples $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$
- 5: Update the discriminator via SGD:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \nabla_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\boldsymbol{\theta}}(\mathbf{x}_i) + \log(1 - D_{\boldsymbol{\theta}}(G_{\boldsymbol{\theta}}(\mathbf{z}_i))) \right]$$

- 6: **end for**
- 7: Sample minibatch of m noise samples $\{\mathbf{z}_1, \dots, \mathbf{z}_m\}$ from $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- 8: Update the generator via SGD:

$$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \eta \nabla_{\boldsymbol{\phi}} \frac{1}{m} \sum_{i=1}^m \left[\log(1 - D_{\boldsymbol{\theta}}(G_{\boldsymbol{\phi}}(\mathbf{z}_i))) \right]$$

- 9: **end for**
-

Remark: code in blue is for updating generator while code in red is for updating discriminator.

Further notes of Algorithm 1

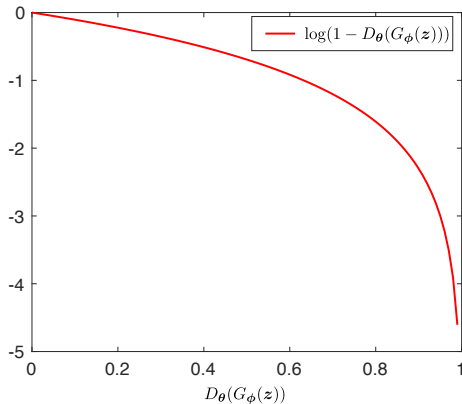
- In Algorithm 1, the discriminator is usually updated multiple ($k > 1$) times for each generator update. By doing so, the feedback provided from the discriminator to the generator is more reliable when updating the generator.
- The discriminator is updated by performing gradient ascent (because of maximization in (1)) while the generator is updated by performing gradient descent (because of minimization (1)).

Convergence of GAN

- The training process converges when a Nash-equilibrium (game theory concept) is reached in the minmax (zero-sum) game.
- In game theory, a Nash-Equilibrium is reached when the actions of one player won't change given the opponent's actions.
- In the GAN framework, it means that the generator produces realistic fake samples that are indistinguishable from real samples.
- Consequently, the discriminator outputs random predictions (probabilities close to 0.5).

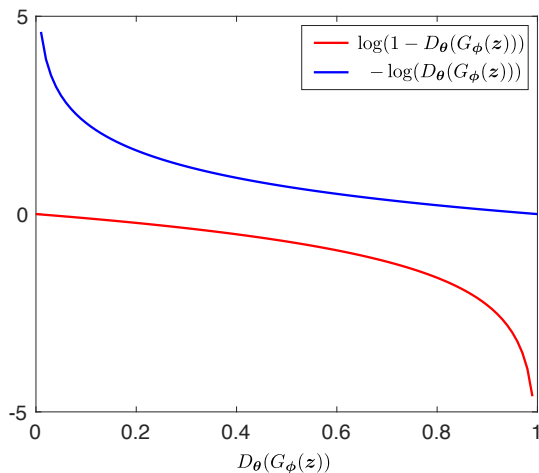
Inefficiency of Algorithm 1

- The function $\log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))$ is not informative when updating the generator.



Remark: The gradient of $\log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))$ for a small $D_{\theta}(G_{\phi}(\mathbf{z}))$ value is small, making the training procedure slow.

Alternative update expression for Generator



Remark: The gradient of $-\log(D_{\theta}(G_{\phi}(\mathbf{z})))$ for a small $D_{\theta}(G_{\phi}(\mathbf{z}))$ value is large, making the training procedure relatively fast.

Training procedure

Algorithm 2 Modified GAN Training

- 1: **for** number of training iterations **do**
- 2: **for** for k steps **do**
- 3: Sample minibatch of m noise samples $\{\mathbf{z}_1, \dots, \mathbf{z}_m\}$ from $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- 4: Sample minibatch of m real data samples $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$
- 5: Update the discriminator via SGD:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \nabla_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\boldsymbol{\theta}}(\mathbf{x}_i) + \log(1 - D_{\boldsymbol{\theta}}(G_{\boldsymbol{\theta}}(\mathbf{z}_i))) \right]$$

- 6: **end for**
- 7: Sample minibatch of m noise samples $\{\mathbf{z}_1, \dots, \mathbf{z}_m\}$ from $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- 8: Update the generator via SGD:

$$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \eta \nabla_{\boldsymbol{\phi}} \frac{1}{m} \sum_{i=1}^m \left[\log(D_{\boldsymbol{\theta}}(G_{\boldsymbol{\phi}}(\mathbf{z}_i))) \right]$$

- 9: **end for**
-

Remark: Differently from that of Algorithm 1, the generator is updated by maximizing $\frac{1}{m} \sum_{i=1}^m$

GAN Training Problems

- Oscillation between generator and discriminator loss.
- Mode collapse (generator produces examples of a particular kind only).
- Discriminator is too strong, such that the gradient for the generator vanishes and the generator can't keep up.
- Discriminator is too weak, and the generator produces nonrealistic samples that fool it too easily (rare problem, though).

References

- [blog of Alexia Jolicoeur-Martineau](#)
- [StyleGAN — Official TensorFlow Implementation](#)
- [Fake Face Generator](#)
- [Lecture notes by Mitesh M. Khapra](#)
- [Lecture notes by Sebastian Raschka](#)

Questions?